NOVA IMS INFORMATION MANAGEMENT SCHOOL

if: one word, two paths

Computation I · Lecture 05

Karina Brotto Rebuli
krebuli@novaims.unl.pt

The crash from last lecture

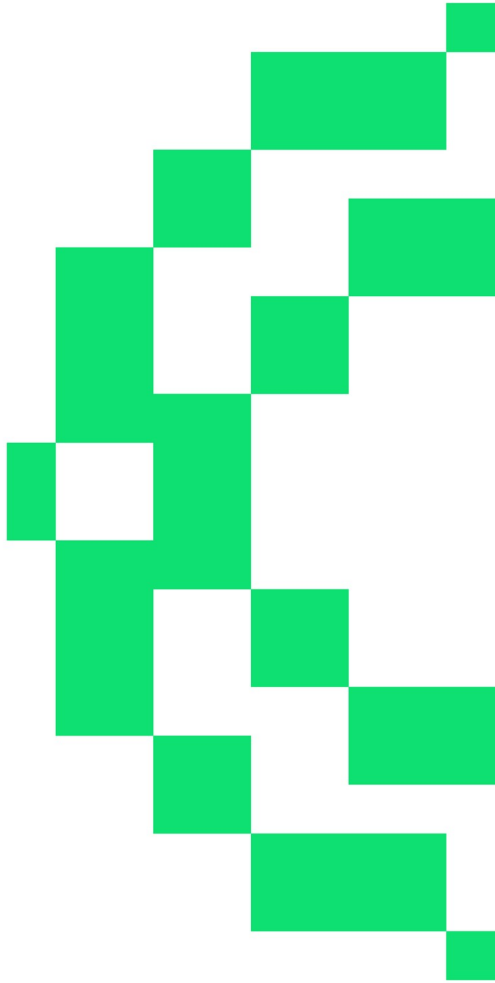
Exercise 10, replayed — answer 20, then twenty

```
age = int(input("Age: "))
print(age + 1)
```

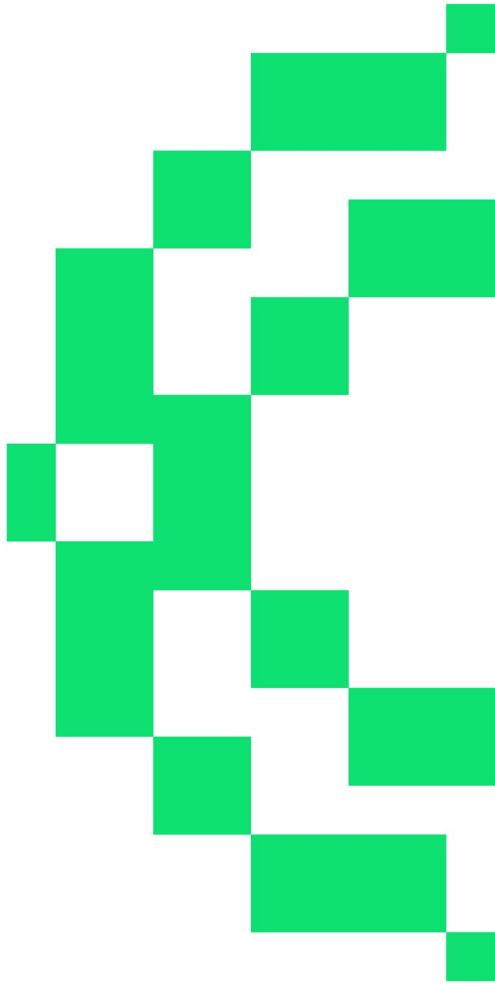
```
Age: 20
21
Age: twenty
ValueError: invalid literal for int() with base 10: 'twenty'
```

One path only

Whatever you typed, the same line ran. Nothing was checked, nothing was chosen. What was missing was a way to decide.



```
"""True runs the block. False walks past it."""
```



George Boole

1854 — Every condition you write today ends as one of his two values.

The problem: an argument was sound or it was not — but soundness lived in words, and words argue back. He wanted the truth of a sentence to be something a person could calculate, the way a sum is calculated.

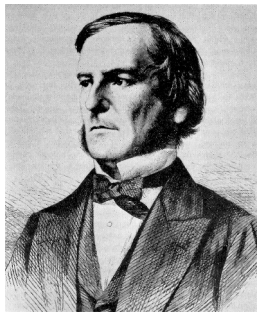


Image source

Three questions today

How a program decides.

How many paths it can take.

How to guard the door.

01.

The program decides

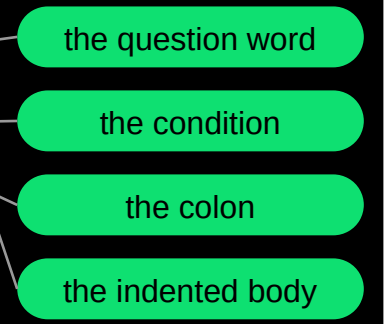
The choosing word

Anatomy of the if

A condition, a colon, an indented block — the indentation decides what belongs

THE PARTS:

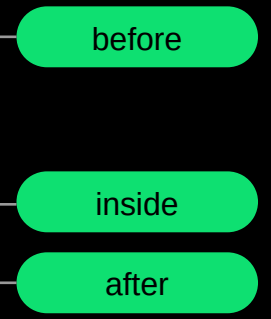
```
grade = 12
if grade >= 10:
    print("Passed")
```




```
>>> Passed
```

THE FLOW: BEFORE, [INSIDE,] AFTER.

```
age = 20
print("Before")
if age >= 18:
    print("Can vote")
print("End")
```




```
>>> Before
>>> Can vote
>>> End
```

LIVE

The program takes a branch

Same program, two runs — True runs the block, False walks past it

THE CONDITION IS **TRUE**:

```
age = 20
if age >= 18:
    print("Adult")
print("Done")
```

```
>>> Adult
>>> Done
```

THE CONDITION IS **FALSE**:

```
age = 15
if age >= 18:
    print("Adult")
print("Done")
```

```
>>> Done
```

else: the second path

When False also has work — exactly one block runs

```
grade = 8
if grade >= 10:
    print("Passed")
else:
    print("Failed")
print("Recorded")
```

```
>>> Failed
>>> Recorded
```

Which one to choose — if vs. if-else

Count the cases you have to answer: one case needs only if, two need else as well

CASE 01

`if`

grade ≥ 10	**pass**
everything else	nothing to do

Congratulate the students who passed.

Use it when only one case needs an action and the rest can be left alone.

CASE 02

`if · else`

grade ≥ 10	**pass**
everything else	**fail**

…and tell the others they failed.

Use it when there are two cases and both need an action.

LIVE

True is True?

The variable already contains the bool result, so comparing it with True is redundant

THIS IS TO BE AVOIDED:

```
age = 20
check_age = age >= 18
if check_age == True:
    print("Adult")
print("Done")
```

```
>>> Adult
>>> Done
```

THIS IS OK:

```
age = 20
check_age = age >= 18
if check_age:
    print("Adult")
print("Done")
```

```
>>> Adult
>>> Done
```

The lie, on the table

== is exact — case included

```
answer = input("Do you confirm? (yes/no) ")
print(answer == "yes")
```

```
Do you confirm? (yes/no) yes
True
Do you confirm? (yes/no) Yes
False
```

LIVE

The lie, on the table

== is exact — case included

```
answer = input("Continue? yes or no: ")
if answer.lower() == "yes":
    print("Continuing.")
else:
    print("Stopped.")
```

```
Continue? yes or no: Yes
Continuing.
```

What does this print?

```
grade = 19
if grade >= 10:
    print("Passed")
if grade >= 18:
    print("Distinction")
```

```
>>> Passed
>>> Distinction
```

02.

Many paths

The first True condition wins

if-else inside the else vs. elif

Both answer every grade in the same way; only the shape of the decision changes

CASE 03 · NESTED

`if · else → if · else`

- grade ≥ 18 — **distinction**
- otherwise
  - grade ≥ 10 — **pass**
  - everything else — **fail**

The second decision is taken inside the else of the first. A fourth case is one more level of indentation.

CASE 03 · FLAT

`if · elif · else`

- grade ≥ 18 — **distinction**
- otherwise, grade ≥ 10 — **pass**
- everything else — **fail**

Three tests, all at the same level, read from the top down. A fourth case is one more row.

LIVE

elif: else if, fused

The chain removes the nested structure and asks in order — the first True wins

```
grade = 19
if grade < 10:
    print("Failed")
elif grade < 18:
    print("Passed")
else:
    print("Distinction")
```

```
>>> Distinction
```

Write range comparison directly, as in mathematics

It is equivalent to 0 <= g and g <= 20 for this variable, but with just one evaluation of g

```
g = int(input("Grade: "))
if 0 <= g <= 20:
    print("Valid grade")
else:
    print("Out of range")
```

```
Grade: 20
Valid grade
```

LIVE

Nest or chain?

Some questions depend on earlier answers — the guard has to come first

THE NEST:

```
s = input("Write a word: ")
if len(s) > 0:
    if s[0] == "a":
        print("Starts with a")
else:
    print("Empty word")
```

```
Write a word: after
Starts with a
```

THE SAME, FLAT:

```
s = input("Write a word: ")
if len(s) == 0:
    print("Empty word")
elif s[0] == "a":
    print("Starts with a")
```

```
Write a word: after
Starts with a
```

When flat costs you

The chain still gets there — by writing the same line twice

TWO COPIES OF THE LINE:

```
s = input("Write a word: ")
if len(s) > 0 and s[0] == "a":
    print("Word received")
    print("Starts with a")
elif len(s) > 0:
    print("Word received")
```

```
Write a word: after
Word received
Starts with a
```

ONE COPY OF THE LINE:

```
s = input("Write a word: ")
if len(s) > 0:
    print("Word received")
    if s[0] == "a":
        print("Starts with a")
```

```
Write a word: after
Word received
Starts with a
```

Where is this one broken?

Written to greet the a-words and to refuse an empty answer

```
s = "banana"
if len(s) > 0:
    if s[0] == "a":
        print("Starts with a")
    else:
        print("Empty word")
```

```
>>> Empty word
```

03.

Guarding the door

Ask before you convert

The crash, on the table

int() believed, then stopped

```
age = int(input("Age: "))
print(age + 1)
```

```
Age: twenty
ValueError: invalid literal for int() with base 10: 'twenty'
```

LIVE

Ask first, convert after

isdigit() answers whether every character is a digit

```
s = input("Age: ")
if s.isdigit():
    age = int(s)
    print(age + 1)
else:
    print("That was not a valid age.")
```

```
Age: 20.5
That was not a valid age.
```

The guard has a known blind spot

The dot is not a digit

```
print("35".isdigit())
print("3.5".isdigit())
```

```
>>> True
>>> False
```

Patch the blind spot

Hide one dot, then ask

```
x = "3.5"
print( x.replace(".", "", 1) )
print( x.replace(".", "", 1).isdigit() )
```

```
>>> 35
>>> True
```

Is there a dot in it?

Week 3's `in` answers it — a float carries a dot, an int does not

```
x = "3.5"
if x.replace(".", "", 1).isdigit():
    print("." in x)
```

```
>>> True
```

int or float, behind the guard

The nest tells them apart

```
x = input("Write a number: ")
if x.replace(".", "", 1).isdigit():
    if "." in x:
        print("This is a float")
    else:
        print("This is an int")
```

```
Write a number: 20.5
This is a float
```

The square, safely with int

Last lecture's square, guarded

```python
x = input("Write an integer: ")
if x.isdigit():
    print(f"The square of {x} is {int(x) ** 2}")
else:
    print("You did not write an integer.")
```

```
Write an integer: 5.5
You did not write an integer.
```

LIVE

The square, safely with float

Last lecture's square, guarded

```python
x = input("Write a number: ")
if x.replace(".", "", 1).isdigit():
    print(f"The square of {x} is {float(x) ** 2}")
else:
    print("You did not write a valid number.")
```

```
Write a number: 5.5
The square of 5.5 is 30.25
```

LIVE

The four kinds

The nest moves into the first block — and the bool asks for the exact spellings

```
user_value = input("Write your data: ")
if user_value.replace(".", "", 1).isdigit():
    if "." in user_value:
        print("This is a float")
    else:
        print("This is an int")
elif user_value in ["True", "False"]:
    print("This is a bool")
else:
    print("This is a str")
```

Three questions, answered

In one sentence each

A program decides

with if — True runs the block, False walks past it; with an else, exactly one branch always runs

The first True condition wins

elif asks in order

The door is guarded

isdigit answers before convertion runs — convert only inside the True

George Boole

1854

Every branch you wrote today was taken on one of two values.

Nothing else was ever needed.

Before the next theoretical lecture

On your own

Think Python — Sections 5.4 `if` statements · 5.5 The `else` clause · 5.6 Chained conditionals · 5.7 Nested Conditionals · 5.12 Debugging.

One habit — before you run an if, read its condition aloud and answer `True or False` yourself.

Exercise 1 · Predict what it prints

Read the program below and write exactly what it prints. On paper, before any run. Then run it and compare.

```
x = 3
if x > 10:
    print("Big")
```

Exercise 2 · Predict what it prints

Read the program below and answer on paper: which of the two paths runs, and what does it print? Then run it and compare.

```
word = "if"
if len(word) > 3:
    print("Long word")
else:
    print("Short word")
```

Exercise 3 · Fix the program

This program should print Passed. Python refuses to run it — it stops before the first line executes. Read the message, name the family of the error, and fix the program.

```
grade = 15
if grade >= 10:
print("Passed")
```

Exercise 4 · Predict, then fix

Read the chain below: it prints whether a student passed or failed. Assume that only grades from 0 to 20 should exist, and that a student fails with a grade below 10. On paper, before any run: which line prints for g = 10, and is that right? Fix what is wrong, then say which line prints for g = 20. Mind the ends of each range.

```
g = 10
if g < 0:
    print("Invalid")
elif 0 <= g <= 10:
    print("Failed")
elif 10 <= g < 20:
    print("Passed")
else:
    print("Distinction")
```

Exercise 5 · Even or odd, by length

Write the rest of the program: print even if the string the user types has an even number of characters, odd otherwise — spaces at the ends stripped first. Start from the input line given in the panel. Use this lecture's if / else, with strip and len from week 3 and % from week 2.

```
s = input("Write a string: ")
# your lines here
```

Exercise 6 · Sell tickets, safely

Write the rest of the program: tickets are €7 each — print what the user pays, but only if what was typed is really a whole number; otherwise, say so. Write it twice: one that stops after a wrong answer, one that asks again. Start from the input line in the panel, and use this lecture's guard — ask first, convert after.

```
t = input("How many tickets? ")
# your lines here
```

Exercise 7 · One separator only

Write the rest of the program: if the date the user typed uses hyphens, swap every hyphen for a slash; then print the date, either way. Start from the input line in the panel. Use this lecture's if, together with in and replace from week 3.

```
date = input("Date: ")
# your lines here
```

Exercise 9 · One letter per line ★

Run the program — one letter per line, and it works for the word grade. Now make it work for a word the user types, whatever its length, using only what has been taught so far.

```
word = "grade"
print(word[0])
print(word[1])
print(word[2])
print(word[3])
print(word[4])
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

Portugal 2030 · Lisboa 2030 · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026